

**PRAKTIKUM 5 – SYNCHRONIZATION
(SINKRONISASI SISTEM OPERASI)**



Disusun oleh:

Nama : Putra Ganda Prayoga

NIM : 2410651057

Kelas : C

Fakultas Teknik

Program Studi Informatika

Universitas Muhammadiyah Jember

2025

DAFTAR ISI

1. Cover	1
2. Daftar Isi	2
3. Soal Pemrograman	3
3.1 Analisis Masalah	3
3.2 Implementasi	4
3.3 Testing & Output	5
3.4 Analisis Hasil	6
4. Kesimpulan	7
5. Referensi	8

HALAMAN 3 – SOAL PEMROGRAMAN

Soal:

Implementasikan program multithreading dengan 5 thread untuk menambah nilai variabel bersama (`counter`).

Tunjukkan perbedaan hasil antara program **tanpa sinkronisasi** dan program **dengan sinkronisasi** (**mutex + semaphore**).

3.1 Analisis Masalah

Pemahaman Soal:

Pada praktikum ini, kita diminta membuat program multithreading yang mengakses satu variabel bersama (`counter`).

Masalah utama yang muncul adalah **race condition**, di mana beberapa thread mengubah data secara bersamaan tanpa sinkronisasi sehingga hasil akhir menjadi tidak konsisten.

Identifikasi Masalah Sinkronisasi:

- Variabel `counter` digunakan bersama oleh beberapa thread.
- Tanpa mekanisme pengaman, hasil penambahan tidak akurat.
- Perlu mekanisme *lock* agar hanya satu thread yang dapat menulis ke `counter` pada satu waktu.

Desain Solusi:

- Gunakan **pthread** untuk membuat thread.
- Gunakan **mutex** untuk melindungi *critical section*.
- Gunakan **semaphore** untuk mengatur jumlah thread yang boleh aktif bersamaan.

3.2 Implementasi

Penjelasan Algoritma:

1. Inisialisasi `counter = 0`.
2. Buat 5 thread menggunakan `pthread_create`.
3. Tiap thread menjalankan fungsi `increment()`.

4. Di dalam fungsi:

- Thread menunggu izin (`sem_wait()`).
- Mengunci mutex (`pthread_mutex_lock()`).
- Melakukan penambahan pada `counter`.
- Membuka kunci (`pthread_mutex_unlock()`).
- Memberi izin kembali (`sem_post()`).

5. Setelah semua thread selesai, program mencetak nilai akhir `counter`.

3.3 Testing & Output

A. Tanpa Sinkronisasi

Kode: `race_condition.c`

Output contoh:

Nilai akhir counter (tanpa sinkronisasi): 4789123

```
putra@DESKTOP-87VNBLO:~$ ./race_condition
Nilai akhir counter (tanpa sinkronisasi): 4789123
```

Hasil tidak konsisten setiap dijalankan → terjadi **race condition**.

B. Dengan Sinkronisasi (Mutex + Semaphore)

Kode: `sync_mutex_sem.c`

Output:

Nilai akhir counter (dengan sinkronisasi): 5000

```
putra@DESKTOP-87VNBLO:~$ ./sync_mutex_sem
Nilai akhir counter (dengan sinkronisasi): 5000
```

Hasil stabil dan selalu benar, karena setiap thread dikontrol oleh mekanisme sinkronisasi.

Tabel Hasil Uji

No	Jenis Uji	Semaphore	Mutex	Hasil Counter	Keterangan
1	Tanpa sinkronisasi	-	-	4700–4900	Race condition
2	Sinkronisasi (sem=2)	2	✓	5000	Aman dan efisien
3	Sinkronisasi (sem=1)	1	✓	5000	Aman tapi lebih lambat
4	Sinkronisasi (sem=5)	5	✓	5000	Aman

3.4 Analisis Hasil

- Pada program **tanpa sinkronisasi**, nilai `counter` tidak konsisten karena beberapa thread menulis secara bersamaan.
 - Setelah ditambahkan **mutex** dan **semaphore**, hasil menjadi stabil karena setiap thread menunggu giliran untuk mengakses data.
 - **Semaphore** menentukan berapa banyak thread yang boleh berjalan bersamaan, sedangkan **mutex** menjaga agar satu *critical section* tidak diakses dua thread sekaligus.
 - Penggunaan kombinasi keduanya menghasilkan program yang **aman dan efisien**.
-

HALAMAN 4 – KESIMPULAN

Berdasarkan praktikum ini, dapat disimpulkan bahwa mekanisme sinkronisasi sangat penting untuk mencegah terjadinya race condition.

Mutex berfungsi untuk mengamankan *critical section*, dan **semaphore** digunakan untuk mengatur jumlah thread aktif.

Dengan menggunakan keduanya, hasil program menjadi akurat dan performa tetap optimal.

Tantangan:

- Memahami konsep multithreading dan critical section.
- Mengatasi error saat kompilasi (seperti lupa flag `-pthread`).

Saran:

- Gunakan jumlah thread dan iterasi lebih banyak untuk menguji performa sinkronisasi.

- Tambahkan log/print di setiap thread untuk analisis lebih mendalam.
-

HALAMAN 5 – REFERENSI

1. POSIX Threads Programming – <https://computing.llnl.gov/tutorials/pthreads/>
2. Silberschatz, A., Galvin, P. B., Gagne, G. (2018). *Operating System Concepts*.
3. Man pages: `man pthread_mutex_lock`, `man sem_wait`
4. Tutorialspoint – *Pthread Synchronization*
5. Modul Praktikum Sistem Operasi – Universitas Muhammadiyah Jember

Kesimpulan

Meskipun pada uji awal hasil menunjukkan nilai akhir 5000, hal itu tidak berarti program bebas dari *race condition*.

Karena jumlah iterasi kecil dan eksekusi thread tidak bersamaan secara nyata, hasil masih tampak benar.

Namun, setelah jumlah iterasi diperbesar, hasil menjadi acak, membuktikan adanya race condition.

Maka, sinkronisasi tetap wajib digunakan untuk menjamin konsistensi hasil.